

Practical 1: Installing and setting environment variables for Working with Apache Hadoop

Aim

To configure Hadoop environment variables and verify Hadoop installation.

Step 1: Download and Verify Hadoop Installation

Download Apache Hadoop tar.gz from the website and extract To check Hadoop directory and confirm installation.

```
%%bash
ls ~/Msc_Practical/big_data_practical/software/hadoop/

hadoop-3.4.2
hadoop-3.4.2.tar.gz
```

Step 2: Set Hadoop Environment Variables

Hadoop requires:

- `HADOOP_HOME` → points to Hadoop installation directory
- `PATH` → allows Hadoop commands to run globally
- `JAVA_HOME` → configured in system

```
%%bash
export HADOOP_HOME=~/Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2
export PATH=$PATH:$HADOOP_HOME/bin
echo $HADOOP_HOME

/home/vbmain/Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2
```

Step 3: Configure Hadoop Environment Files

Hadoop requires configuration in XML files:

- `hadoop-env.sh` → sets `JAVA_HOME`
- `core-site.xml` → defines filesystem
- `hdfs-site.xml` → HDFS storage settings
- `mapred-site.xml` → MapReduce framework settings

Set `JAVA_HOME` in Hadoop

```
%%bash
HADOOP_HOME=~/Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2

sed -i 's|# export JAVA_HOME=.*|export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64|' \
$HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

```
%%bash
grep JAVA_HOME ~/Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2/etc/hadoop/hadoop-env.sh

# JAVA_HOME=/usr/java/testing hdfs dfs -ls
# Technically, the only required environment variable is JAVA_HOME.
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

Step 4: Configure core-site.xml

Define the Hadoop default filesystem (HDFS URI).

`core-site.xml` tells Hadoop where the NameNode is located.
Configure local pseudo-distributed mode using:

hdfs://localhost:9000

```
%%bash
HADOOP_HOME=~/.Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2

cat > $HADOOP_HOME/etc/hadoop/core-site.xml << EOF
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
EOF
```

```
%%bash
cat ~/.Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2/etc/hadoop/core-site.xml

<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

Step 5: Configure hdfs-site.xml

Define HDFS storage configuration including replication factor and data directory.

`hdfs-site.xml` controls HDFS behavior:

- replication factor (for single-node setup = 1)
- NameNode and DataNode storage paths

```
%%bash
HADOOP_HOME=~/.Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2

cat > $HADOOP_HOME/etc/hadoop/hdfs-site.xml << EOF
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>

  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///tmp/hadoop-namenode</value>
  </property>

  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///tmp/hadoop-datanode</value>
  </property>
</configuration>
EOF
```

```
%%bash
cat ~/.Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2/etc/hadoop/hdfs-site.xml
```

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>

  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///tmp/hadoop-namenode</value>
  </property>

  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///tmp/hadoop-datanode</value>
  </property>
</configuration>
```

Step 6: Configure mapred-site.xml

Configure Hadoop to use the YARN framework for executing MapReduce jobs.

`mapred-site.xml` specifies the MapReduce execution framework.

```
%%bash
HADOOP_HOME=~/.Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2

cat > $HADOOP_HOME/etc/hadoop/mapred-site.xml << EOF
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
EOF
```

```
%%bash
cat ~/.Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2/etc/hadoop/mapred-site.xml

<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

Step 7: Verify Hadoop Configuration

To verify that Hadoop is correctly configured and ready for use.

```
%%bash
~/Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2/bin/hadoop version

Hadoop 3.4.2
Source code repository https://github.com/apache/hadoop.git -r 84e8b89ee2ebe6923691205b9e171badde7a495c
Compiled by ahmarsu on 2025-08-20T10:30Z
Compiled on platform linux-x86_64
Compiled with protoc 3.23.4
From source with checksum fa94c67d4b4be021b9e9515c9b0f7b6
This command was run using /home/vbmain/Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2/share/hadoop
p/common/hadoop-common-3.4.2.jar
```

Step 8: Complete Hadoop Runtime Configuration

Configure YARN

Edit `yarn-site.xml` and add:

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
```

```
</configuration>
```

Configure MapReduce

Edit `mapred-site.xml` and add:

```
<configuration>

  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>

  <property>
    <name>yarn.app.mapreduce.am.env</name>
    <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
  </property>

  <property>
    <name>mapreduce.map.env</name>
    <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
  </property>

  <property>
    <name>mapreduce.reduce.env</name>
    <value>HADOOP_MAPRED_HOME=${HADOOP_HOME}</value>
  </property>

</configuration>
```

Install and Configure OpenSSH

```
sudo apt update
sudo apt install openssh-server -y
```

```
sudo systemctl enable ssh
sudo systemctl start ssh
```

Configure passwordless SSH:

```
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa

cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys

chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

Format NameNode (First Time Only)

```
hdfs namenode -format
```

Start Hadoop

```
start_hadoop
```

Verify:

```
jps
```

Expected processes:

- NameNode
- DataNode
- SecondaryNameNode
- ResourceManager
- NodeManager

Result

Apache Hadoop was successfully configured by setting the required environment variables and updating the Hadoop

configuration files. The installation was verified using Hadoop commands.

Conclusion

The Hadoop environment was configured successfully and is ready for running HDFS and MapReduce applications.

Practical 2: Implementing Map-Reduce Program for Word Count problem

Aim

To implement the Word Count problem using Hadoop Streaming with Python.

Theory

Word Count is a basic MapReduce application used to count the occurrences of each word in a text file. The Mapper emits each word with a count of 1, and the Reducer sums the counts for each word.

Step 1: Create Input File

Create a sample text file for the Word Count program.

```
from pathlib import Path

project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

dataset_dir = project_dir / "datasets"
dataset_dir.mkdir(exist_ok=True)

input_file = dataset_dir / "wordcount.txt"

input_file.write_text(
    """Hadoop is Big Data
    Big Data is Powerful
    Hadoop Hadoop Spark
    """,
    encoding="utf-8"
)

print(input_file)
print(input_file.read_text())
```

```
/home/vbmain/Msc_Practical/big_data_practical/datasets/wordcount.txt
Hadoop is Big Data
Big Data is Powerful
Hadoop Hadoop Spark
```

Step 2: Create the Mapper

The Mapper reads each line from the input file and emits every word with a count of 1.

```
from pathlib import Path

project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

mapper_dir = project_dir / "mapper"
mapper_dir.mkdir(exist_ok=True)

mapper_file = mapper_dir / "mapper.py"

mapper_code = """#!/usr/bin/env python3

import sys

for line in sys.stdin:
    for word in line.strip().split():
        print(f"{word}\\t1")
"""
```

```

mapper_file.write_text(mapper_code, encoding="utf-8")

print(f"Created: {mapper_file}")
print()
print(mapper_file.read_text())

Created: /home/vbmain/Msc_Practical/big_data_practical/mapper/mapper.py

#!/usr/bin/env python3

import sys

for line in sys.stdin:
    for word in line.strip().split():
        print(f"{word}\t1")

%%bash
chmod +x ../mapper/mapper.py

```

Step 3: Create the Reducer

The Reducer reads the Mapper output, groups identical words, and calculates the total count for each word.

```

from pathlib import Path

project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

reducer_dir = project_dir / "reducer"
reducer_dir.mkdir(exist_ok=True)

reducer_file = reducer_dir / "reducer.py"

reducer_code = """#!/usr/bin/env python3

import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.strip().split("\t")
    count = int(count)

    if current_word == word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")
            current_word = word
            current_count = count

if current_word is not None:
    print(f"{current_word}\t{current_count}")
"""

reducer_file.write_text(reducer_code, encoding="utf-8")

print(f"Created: {reducer_file}")
print()
print(reducer_file.read_text())

```

```
Created: /home/vbmain/Msc_Practical/big_data_practical/reducer/reducer.py
```

```
#!/usr/bin/env python3

import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.strip().split("\t")
    count = int(count)

    if current_word == word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")
        current_word = word
        current_count = count

if current_word is not None:
    print(f"{current_word}\t{current_count}")
```

```
%%bash
chmod +x ../reducer/reducer.py
```

Step 4: Test the MapReduce Program Locally

To verify the Mapper and Reducer before executing them using Hadoop Streaming.

```
%%bash
cat ../datasets/wordcount.txt \
| python3 ../mapper/mapper.py \
| sort \
| python3 ../reducer/reducer.py

Big      2
Data     2
Hadoop   3
Powerful 1
Spark    1
is       2
```

Step 5: Execute the Word Count Program using Hadoop Streaming

To execute the Word Count MapReduce program on Hadoop using Python Streaming.

```
%%bash
source ../config.sh

STREAMING_JAR=$(find "$HADOOP_HOME/share/hadoop/tools/lib" -name "hadoop-streaming-*.jar" | head -n 1)

hdfs dfs -mkdir -p /input
hdfs dfs -put -f ../datasets/wordcount.txt /input/

hdfs dfs -rm -r -f /output

hadoop jar "$STREAMING_JAR" \
  -files ../mapper/mapper.py,../reducer/reducer.py \
  -input /input/wordcount.txt \
  -output /output \
  -mapper "python3 mapper.py" \
  -reducer "python3 reducer.py"
```



```

=====
Big Data Practical Environment Loaded
=====
Project Home : /home/vbmain/Msc_Practical/big_data_practical
Python Venv  : /home/vbmain/Msc_Practical/big_data_practical/venv
Java Home    : /usr/lib/jvm/java-17-openjdk-amd64
Hadoop Home  : /home/vbmain/Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2
Spark Home   : /home/vbmain/Msc_Practical/big_data_practical/software/spark/spark-3.5.7-bin-hadoop3
HBase Home   : /home/vbmain/Msc_Practical/big_data_practical/software/hbase/hbase-2.6.3
Pig Home     : /home/vbmain/Msc_Practical/big_data_practical/software/pig/pig-0.18.0
Sqoop Home   : /home/vbmain/Msc_Practical/big_data_practical/software/sqoop/sqoop-1.4.7.bin__hadoop-2.6.0
Flume Home   : /home/vbmain/Msc_Practical/big_data_practical/software/flume/apache-flume-1.11.0-bin
Helper Scripts : /home/vbmain/Msc_Practical/big_data_practical/bin
=====

Deleted /output
packageJobJar: [/tmp/hadoop-unjar4955788590810607093/] [] /tmp/streamjob8540763503736373664.jar tmpDir=null

```

Step 6: View the Output

Display the output generated by the Hadoop Streaming job.

```

%bash
source ../config.sh

hdfs dfs -cat /output/part-00000

```

```

=====
Big Data Practical Environment Loaded
=====
Project Home : /home/vbmain/Msc_Practical/big_data_practical
Python Venv  : /home/vbmain/Msc_Practical/big_data_practical/venv
Java Home    : /usr/lib/jvm/java-17-openjdk-amd64
Hadoop Home  : /home/vbmain/Msc_Practical/big_data_practical/software/hadoop/hadoop-3.4.2
Spark Home   : /home/vbmain/Msc_Practical/big_data_practical/software/spark/spark-3.5.7-bin-hadoop3
HBase Home   : /home/vbmain/Msc_Practical/big_data_practical/software/hbase/hbase-2.6.3
Pig Home     : /home/vbmain/Msc_Practical/big_data_practical/software/pig/pig-0.18.0
Sqoop Home   : /home/vbmain/Msc_Practical/big_data_practical/software/sqoop/sqoop-1.4.7.bin__hadoop-2.6.0
Flume Home   : /home/vbmain/Msc_Practical/big_data_practical/software/flume/apache-flume-1.11.0-bin
Helper Scripts : /home/vbmain/Msc_Practical/big_data_practical/bin
=====

Big      2
Data     2
Hadoop   3
Powerful      1
Spark    1
is       2

```

Result

The Word Count MapReduce program was successfully implemented using Hadoop Streaming with Python. The output correctly displayed the frequency of each word in the input file.

Conclusion

The practical demonstrated the execution of a Hadoop MapReduce application using Python mapper and reducer scripts. Hadoop Streaming enables Python programs to be used instead of Java for MapReduce processing.

Practical 3: Write a program to Implement a tri-gram model

Aim

To implement a Tri-gram language model using Python and NLTK.

Theory

A Tri-gram model is an n-gram language model in which every sequence consists of three consecutive words. It is commonly used in Natural Language Processing for language modeling, text prediction, and machine translation.

Step 1: Create a sample text dataset for generating tri-grams.

```
from pathlib import Path

project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

dataset_dir = project_dir / "datasets"
dataset_dir.mkdir(exist_ok=True)

dataset = dataset_dir / "trigram.txt"

dataset.write_text(
    """Natural language processing is an important field of artificial intelligence.
    Natural language processing enables computers to understand human language.
    """,
    encoding="utf-8"
)

print(dataset)
print()
print(dataset.read_text())
```

/home/vbmain/Msc_Practical/big_data_practical/datasets/trigram.txt

Natural language processing is an important field of artificial intelligence.
Natural language processing enables computers to understand human language.

Step 2: Read the dataset, tokenize the text, and generate tri-grams using NLTK.

```
from pathlib import Path
from nltk import word_tokenize
from nltk.util import trigrams
import nltk

# Download required resources (only needed the first time)
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)

project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

text = (project_dir / "datasets" / "trigram.txt").read_text(encoding="utf-8")

tokens = word_tokenize(text)

tri_grams = list(trigrams(tokens))

print(f"Number of tokens: {len(tokens)}")
print(f"Number of trigrams: {len(tri_grams)}\n")

# Show only the first 10 trigrams
for tg in tri_grams[:10]:
    print(tg)
```

Number of tokens: 21
Number of trigrams: 19

```
('Natural', 'language', 'processing')
('language', 'processing', 'is')
('processing', 'is', 'an')
('is', 'an', 'important')
('an', 'important', 'field')
('important', 'field', 'of')
('field', 'of', 'artificial')
('of', 'artificial', 'intelligence')
('artificial', 'intelligence', '.')
('intelligence', '.', 'Natural')
```

Step 3: Calculate the frequency of each tri-gram.

```
from collections import Counter
```

```
frequency = Counter(tri_grams)
```

```
print("Tri-gram Frequencies:\n")
```

```
for tg, count in frequency.items():
    print(f"{tg} : {count}")
```

Tri-gram Frequencies:

```
('Natural', 'language', 'processing') : 2
('language', 'processing', 'is') : 1
('processing', 'is', 'an') : 1
('is', 'an', 'important') : 1
('an', 'important', 'field') : 1
('important', 'field', 'of') : 1
('field', 'of', 'artificial') : 1
('of', 'artificial', 'intelligence') : 1
('artificial', 'intelligence', '.') : 1
('intelligence', '.', 'Natural') : 1
('.', 'Natural', 'language') : 1
('language', 'processing', 'enables') : 1
('processing', 'enables', 'computers') : 1
('enables', 'computers', 'to') : 1
('computers', 'to', 'understand') : 1
('to', 'understand', 'human') : 1
('understand', 'human', 'language') : 1
('human', 'language', '.') : 1
```

Step 4: Comparison with spaCy

NLTK	spaCy
Simple and educational	Faster for large datasets
Easy n-gram generation	Better NLP pipeline
Suitable for learning	Suitable for production applications

Result

A Tri-gram model was successfully implemented using NLTK. The generated tri-grams and their frequencies were displayed successfully.

Conclusion

The Tri-gram model demonstrates how sequences of three consecutive words can be generated and analyzed. Such models are widely used in language modeling and text prediction.

Practical 4: Create Graphical data and access the graphical data using Spark

Aim

To create a dataset, process it using PySpark, and visualize the data using a graph.

Theory

Apache Spark is a distributed data processing framework that supports fast, in-memory computation. PySpark allows Spark applications to be written using Python. The processed data can be visualized using Python libraries such as Matplotlib.

Step 1: Create a small CSV dataset for analysis.

```
from pathlib import Path
import pandas as pd

project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

dataset_dir = project_dir / "datasets"
dataset_dir.mkdir(exist_ok=True)

data = pd.DataFrame({
    "Department": ["Sales", "HR", "IT", "Finance", "Marketing"],
    "Employees": [25, 10, 40, 15, 20]
})

csv_file = dataset_dir / "employees.csv"
data.to_csv(csv_file, index=False)

data
```

	Department	Employees
0	Sales	25
1	HR	10
2	IT	40
3	Finance	15
4	Marketing	20

Step 2: Start a local Spark session and load the CSV dataset into a Spark DataFrame.

```
from pathlib import Path
import os

# Locate project directory
project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

# Set Spark environment
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-17-openjdk-amd64"
os.environ["SPARK_HOME"] = str(project_dir / "software" / "spark" / "spark-3.5.7-bin-hadoop3")
os.environ["SPARK_LOCAL_IP"] = "127.0.0.1"

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("Practical4")
    .master("local[*]")
```

```

        .getOrCreate()
    )

    csv_file = project_dir / "datasets" / "employees.csv"

    df = spark.read.csv(str(csv_file), header=True, inferSchema=True)

    df.show()

```

```

+-----+-----+
|Department|Employees|
+-----+-----+
|    Sales|      25|
|      HR|      10|
|      IT|      40|
|  Finance|      15|
|Marketing|      20|
+-----+-----+

```

Step 3: Display departments having more than 15 employees.

```

filtered_df = df.filter(df.Employees > 15)

filtered_df.show()

```

```

+-----+-----+
|Department|Employees|
+-----+-----+
|    Sales|      25|
|      IT|      40|
|Marketing|      20|
+-----+-----+

```

Step 4: Convert the Spark DataFrame to a Pandas DataFrame and visualize the data.

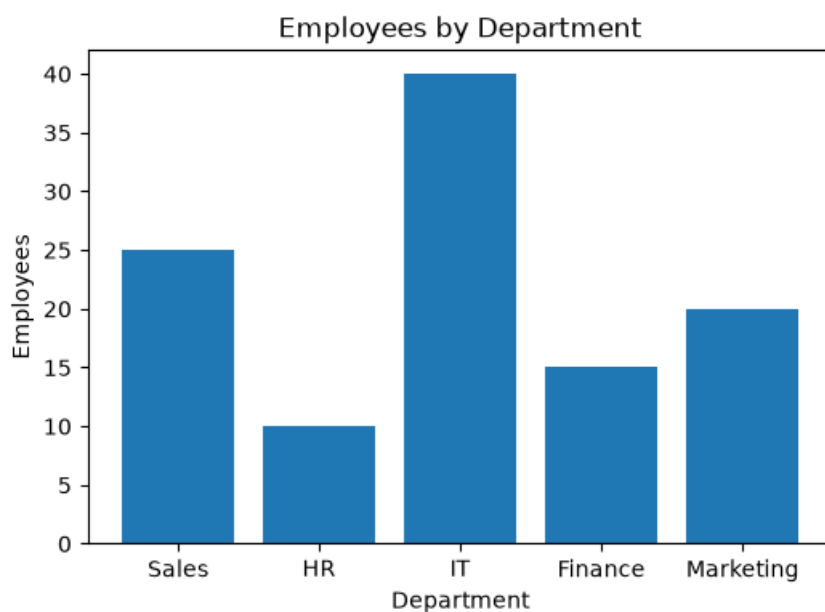
```

import matplotlib.pyplot as plt

pdf = df.toPandas()

plt.figure(figsize=(6, 4))
plt.bar(pdf["Department"], pdf["Employees"])
plt.title("Employees by Department")
plt.xlabel("Department")
plt.ylabel("Employees")
plt.show()

```



Result

The CSV dataset was successfully loaded into a Spark DataFrame, processed using PySpark, and visualized as a bar chart.

Conclusion

PySpark was used to efficiently read and process structured data. The processed data was successfully represented graphically using Matplotlib.

Practical 5: Write a Spark code for the given application and handle error and recovery of data

Aim

To develop a PySpark application with exception handling and demonstrate data recovery.

Theory

Apache Spark provides distributed processing for large datasets. Exception handling improves program reliability by managing runtime errors. Data recovery can be achieved by saving processed data and reloading it when required.

Step 1: Create a sample employee salary dataset.

```
from pathlib import Path
import pandas as pd

project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

dataset_dir = project_dir / "datasets"
dataset_dir.mkdir(exist_ok=True)

employees = pd.DataFrame({
    "Employee": ["Alice", "Bob", "Charlie", "David"],
    "MonthlySalary": [50000, 42000, 60000, 45000]
})

csv_file = dataset_dir / "salary.csv"
employees.to_csv(csv_file, index=False)

employees
```

	Employee	MonthlySalary
0	Alice	50000
1	Bob	42000
2	Charlie	60000
3	David	45000

Step 2: Load the CSV dataset into a Spark DataFrame and calculate the annual salary using PySpark with exception handling.

```
from pathlib import Path
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

os.environ["SPARK_HOME"] = str(
    project_dir / "software" / "spark" / "spark-3.5.7-bin-hadoop3"
)
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-17-openjdk-amd64"
os.environ["SPARK_LOCAL_IP"] = "127.0.0.1"

spark = (
    SparkSession.builder
    .appName("Practical5")
    .master("local[*]")
    .getOrCreate()
```

```

)

try:
    df = spark.read.csv(
        str(project_dir / "datasets" / "salary.csv"),
        header=True,
        inferSchema=True
    )

    df = df.withColumn(
        "AnnualSalary",
        col("MonthlySalary") * 12
    )

    df.show()

except Exception as e:
    print("Error:", e)

```

```

+-----+-----+-----+
|Employee|MonthlySalary|AnnualSalary|
+-----+-----+-----+
|  Alice|      50000|    600000|
|   Bob|      42000|    504000|
| Charlie|      60000|    720000|
|  David|      45000|    540000|
+-----+-----+-----+

```

Step 3: Save the processed data and reload it to demonstrate data recovery.

```

output_dir = project_dir / "outputs" / "salary_output"

# Remove previous output if it exists
import shutil

if output_dir.exists():
    shutil.rmtree(output_dir)

df.write.mode("overwrite").csv(
    str(output_dir),
    header=True
)

recovered_df = spark.read.csv(
    str(output_dir),
    header=True,
    inferSchema=True
)

recovered_df.show()

```

```

+-----+-----+-----+
|Employee|MonthlySalary|AnnualSalary|
+-----+-----+-----+
|  Alice|      50000|    600000|
|   Bob|      42000|    504000|
| Charlie|      60000|    720000|
|  David|      45000|    540000|
+-----+-----+-----+

```

Step 4: Verify that the recovered data matches the processed data.

```

print("Number of records:", recovered_df.count())

recovered_df.printSchema()

Number of records: 4
root
 |-- Employee: string (nullable = true)
 |-- MonthlySalary: integer (nullable = true)
 |-- AnnualSalary: integer (nullable = true)

```


Result

The employee salary dataset was successfully processed using PySpark. Exception handling was implemented, and the processed data was saved and reloaded successfully to demonstrate data recovery.

Conclusion

PySpark simplifies large-scale data processing while Python exception handling improves application reliability. Saving and reloading processed data demonstrates a simple method of data recovery.

Practical 6: Write a Spark code to Handle the Streaming of data

Aim

To implement Spark Structured Streaming using PySpark for processing streaming data.

Theory

Spark Structured Streaming is a scalable stream processing engine built on the Spark SQL API. It processes streaming data as incremental DataFrames and provides fault tolerance with high throughput.

Step 1: Create directories for streaming input and output.

```
from pathlib import Path

project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

stream_input = project_dir / "datasets" / "stream_input"
stream_output = project_dir / "outputs" / "stream_output"

stream_input.mkdir(parents=True, exist_ok=True)
stream_output.mkdir(parents=True, exist_ok=True)

print(stream_input)
print(stream_output)

/home/vbmain/Msc_Practical/big_data_practical/datasets/stream_input
/home/vbmain/Msc_Practical/big_data_practical/outputs/stream_output
```

Step 2: Start a Spark session and create a streaming DataFrame.

```
from pathlib import Path
import os

from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StringType, IntegerType

project_dir = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()

os.environ["SPARK_HOME"] = str(
    project_dir / "software" / "spark" / "spark-3.5.7-bin-hadoop3"
)
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-17-openjdk-amd64"
os.environ["SPARK_LOCAL_IP"] = "127.0.0.1"

spark = (
    SparkSession.builder
    .appName("Practical6")
    .master("local[*]")
    .getOrCreate()
)

schema = (
    StructType()
    .add("Name", StringType())
    .add("Marks", IntegerType())
)

stream_df = (
    spark.readStream
    .schema(schema)
    .option("header", True)
    .csv(str(stream_input))
)
```

```

)

spark.conf.set("spark.sql.adaptive.enabled", "false")

stream_df.printSchema()

root
 |-- Name: string (nullable = true)
 |-- Marks: integer (nullable = true)

```

Step 3: Start the Structured Streaming query.

```

checkpoint_dir = project_dir / "outputs" / "checkpoints"
checkpoint_dir.mkdir(parents=True, exist_ok=True)

# Stop any existing streaming queries
for q in spark.streams.active:
    q.stop()

query = (
    stream_df.writeStream
        .format("console")
        .outputMode("append")
        .option("checkpointLocation", str(checkpoint_dir))
        .start()
)

print("Streaming started.")

```

Streaming started.

Step 4: Add a CSV file to the streaming input directory.

```

import pandas as pd

data = pd.DataFrame({
    "Name": ["Alice", "Bob", "Charlie"],
    "Marks": [85, 92, 78]
})

data.to_csv(
    stream_input / "students.csv",
    index=False
)

print("CSV file added.")

```

CSV file added.

Batch: 0

```

+-----+-----+
|  Name | Marks |
+-----+-----+
|  Alice |   85 |
|   Bob |   92 |
|Charlie |   78 |
+-----+-----+

```

Step 5: Stop the stream

```

query.stop()

print("Streaming stopped.")

```

Streaming stopped.

Result

Spark Structured Streaming successfully monitored the input directory and processed new CSV data automatically.

Conclusion

Spark Structured Streaming enables continuous processing of incoming data with a simple DataFrame-based API. It is suitable for real-time analytics and stream processing.

Practical 7: Apache Flume Log Streaming

Aim

To configure Apache Flume to ingest streaming log data from a local file source and sink it into HDFS (Hadoop Distributed File System).

Step 1: Configure the Flume Agent

Create a configuration file that tells Flume to:

1. Source: Watch a specific log file (spooldir).
2. Channel: Buffer the data in memory.
3. Sink: Write the data into HDFS.

Create the configuration file flume-conf.properties:

```
%%bash
cd ~/Msc_Practical/big_data_practical

cat > flume-conf.properties <<EOF
# Define agent components
agent1.sources = source1
agent1.channels = channel1
agent1.sinks = sink1

# Source
agent1.sources.source1.type = spooldir
agent1.sources.source1.spooldir = /home/vbmain/Msc_Practical/big_data_practical/flume_spool

# Channel
agent1.channels.channel1.type = memory
agent1.channels.channel1.capacity = 1000
agent1.channels.channel1.transactionCapacity = 100

# Sink
agent1.sinks.sink1.type = hdfs
agent1.sinks.sink1.hdfs.path = hdfs://localhost:9000/user/hadoop/flume_logs
agent1.sinks.sink1.hdfs.fileType = DataStream

# Bind components
agent1.sources.source1.channels = channel1
agent1.sinks.sink1.channel = channel1
EOF
```

Step 2: Prepare the Environment

Create the necessary directories for the spooling source and verify HDFS connectivity.

```
%%bash
cd ~/Msc_Practical/big_data_practical

source config.sh

mkdir -p flume_spool

hdfs dfs -mkdir -p /user/hadoop/flume_logs
```

Step 3: Run the Flume Agent

Run the agent in the foreground to observe the data ingestion. (Run in a separate terminal)

```
%%bash
cd ~/Msc_Practical/big_data_practical

export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
source config.sh

$FLUME_HOME/bin/flume-ng agent \
  --conf $FLUME_HOME/conf \
```

```
--conf-file flume-conf.properties \  
--name agent1 \  
-Dflume.root.logger=INFO,console
```

Step 4: Test Ingestion

```
%%bash  
cd ~/Msc_Practical/big_data_practical/  
  
source ~/Msc_Practical/big_data_practical/config.sh  
  
hdfs dfs -ls /user/hadoop/flume_logs  
hdfs dfs -cat /user/hadoop/flume_logs/*  
  
Found 1 items  
-rw-r--r--  1 vbmain supergroup      50 2026-07-01 16:52 /user/hadoop/flume_logs/FlumeData.1782924719465  
Streaming log entry: Wed Jul  1 16:25:19 UTC 2026
```

Result

Apache Flume was successfully configured to ingest streaming log data from a local spool directory and store it in Hadoop Distributed File System (HDFS). The ingested data was verified by reading it back from HDFS, confirming successful end-to-end data ingestion.